

Bahrain House Rental Price Prediction

1 Project: Bahrain House Rental Price Prediction

📁 Projects: Project Overview

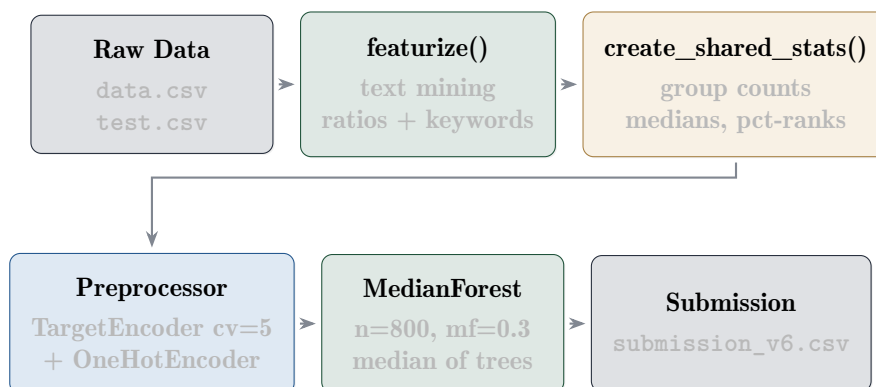
This competition asks you to predict the **monthly rental price of properties in Bahrain**, measured in **BHD** (Bahraini Dinars). The full script loads two CSV files, trains a **Random Forest regressor** on engineered features, and writes a ready-to-submit prediction file called `submission_v6.csv`.

Key ideas at a glance:

- **MedianForest** — a custom `RandomForestRegressor` that predicts the *median* of all tree predictions instead of the mean, because the median is the optimal estimator for **Mean Absolute Error (MAE)**.
- **TargetEncoder** with `cv=5` replaces the earlier manual hierarchical encoding of the categorical columns.
- The target is **capped at 1850 BHD** (outliers removed), but the model is still trained on the *raw* (uncapped) target.
- **Agent_name** is used *only* as a **count** feature — target-encoding it actually **hurt** the score.

1.1 The Full Pipeline

The notebook follows a clean five-stage pipeline: load the raw data, engineer features, target-encode the high-cardinality categoricals, fit the median-optimised Random Forest, and export the predictions. Notice how the preprocessor and model are composed inside a single `sklearn Pipeline`: everything that happens between the raw CSV and the final predictions is one reproducible object.



Pipeline order: raw data → feature engineering → target encoding → MedianForest → submission.

1.2 Custom MAE-Optimized Model: MedianForest

A standard `RandomForestRegressor` averages the predictions of all its trees. But the competition metric is **MAE**, and for MAE the theoretically optimal summary of a set of tree predictions is the **median**, not the mean. The notebook therefore subclasses `RandomForestRegressor` and overrides only `predict()` to aggregate the raw tree predictions with `np.median`:

🔖 Projects: MedianForest — Median Aggregation

For a regression problem scored by **Mean Absolute Error**, the median of the tree predictions is the optimal point estimate: it minimises the expected absolute deviation.

```
class MedianForest(RandomForestRegressor):
    """
    Standard Random Forest predicts the mean of individual
    trees.
    For MAE loss, taking the median across tree predictions is
    optimal.
    """
    def predict(self, X):
        X = np.asarray(X, dtype=np.float32, order='C')
        tree_preds = np.empty((len(self.estimators_), X.shape
                               [0]))

        for i, tree in enumerate(self.estimators_):
            tree_preds[i] = tree.predict(X, check_input=False)

        return np.median(tree_preds, axis=0)
```

Every other behaviour (fitting, feature importance, parameter set) is inherited from `RandomForestRegressor`. Only the aggregation step changes.

The mean squared error is not the median absolute error. Match the metric to your score!

1.3 Feature Engineering

The raw table stores most of the signal inside *free-text* columns (Beds, Baths, Size, Title, URL, Offer). The notebook's `featurize()` function mines numbers and keyword flags out of those strings, builds ratios, and derives availability signals from dates:

✓ Projects: Example — featurize() text mining

```
1 TITLE_PATTERNS = {
2     'sea_view': r'sea\s*view',
3     'brand_new': r'brand\s*new',
4     'luxury': r'luxur',
5     'pool': r'\bpool\b',
6     'gym': r'\bgym\b',
7     'water': r'beach|waterfront|marina|lagoon',
8     'sale': r'for\s*sale|freehold|title\s*deed',
9 }
10
11 def featurize(df):
12     d = df.copy()
13     beds_text = d['Beds'].astype('string').fillna('').str.
14         lower()
15
16     # Extract numerical counts from free text
17     d['beds'] = beds_text.str.extract(r'(\d+\.\d*)', expand=
18         False).astype('float64')
19     d['beds'] = d['beds'].mask(beds_text.str.contains('studio'
20         ), 0.5)
21     d['maid'] = beds_text.str.contains('maid').astype('int8')
22
23     # Size: sqft, sqm and ratios
```

```

21 d['sqft'] = size_text.str.extract(r'([\d,]+)\s*sqft',
22                                expand=False)\
23                                .str.replace(',', '').astype('float64')
24
25 d['sqft_per_bed'] = d['sqft'] / (d['beds'] + 1)
26
27 # Keyword flags mined from the title
28 for name, pattern in TITLE_PATTERNS.items():
29     d[f't_{name}'] = title_text.str.contains(pattern, na=False).astype('int8')
30
31 return d

```

The text mining uses pandas string methods (`.str.extract`, `.str.contains`) with regular expressions, then encodes each finding as a numeric feature the Random Forest can split on.

Feature groups produced by feature engineering:

Feature Group	Examples	Source
Room counts	beds, baths, maid, rooms	Beds / Baths text
Size	sqft, log_sqft, unit_ratio	Size text
Ratios	sqft_per_bed, baths_per_bed, sqft_per_room	engineered
Availability	avail_days, avail_month, avail_dow	Availability_date
Title keywords	t_sea_view, t_luxury, t_pool, t_furn	Title regex flags
Title meta	t_len, t_words, tBars	Title
URL / Offer	url_depth, url_sale, offer_sale	URL / Offer
Shared stats	Area_cnt, Area_med_sqft, rel_sqft, sqft_pct_area	joint train + test groupby
Categorical combos	area_type, area_beds	Area × type / beds
Amenities	amen	Amenities

The `create_shared_stats()` function is a key trick: it concatenates train and test, computes group-level statistics (*counts*, *median sqft*, *percentile ranks*) on the combined table, and then splits them back. This avoids data leakage while giving every listing context about its area, governorate and property type.

1.4 Target Encoding

High-cardinality categorical columns such as **Area** and **Agency** would explode into hundreds of one-hot columns. Instead of the earlier *manual hierarchical encoding* (hand-built groups), the notebook uses scikit-learn's **TargetEncoder** with `cv=5` — the encoder learns a smoothed target mean per category under cross-validation, which is much more robust to overfitting. The preprocessor also keeps a small **OneHotEncoder** for the low-cardinality columns and passes the engineered numeric columns straight through:

Compute group-level statistics on the combined train and test together, then split back. This gives every row the same context about its area, governorate and property type.

🔖 Projects: Preprocessor — TargetEncoder cv=5

```
def build_pipeline():
    te_cols = ['Area', 'area_type', 'area_beds', 'Agency', 'Governorate']
    ohe_cols = ['Property_type', 'Include_w_e', 'Governorate', 'url_type']

    preprocessor = ColumnTransformer(
        transformers=[
            ('te', TargetEncoder(target_type='continuous',
                                smooth='auto', cv=5,
                                random_state=SEED),
            te_cols),
            ('ohe', OneHotEncoder(handle_unknown='infrequent_if_exist',
                                min_frequency=0.004,
                                sparse_output=False),
            ohe_cols),
            ('num', 'passthrough', NUM_COLS)
        ],
        verbose_feature_names_out=False
    )

    model = MedianForest(n_estimators=800, max_features=0.3,
                        min_samples_leaf=1, random_state=SEED,
                        n_jobs=-1)

    return Pipeline([('pre', preprocessor), ('rf', model)])
```

Three transformer groups inside one `ColumnTransformer`: target-encoded categoricals, one-hot categoricals, and untouched numeric features — all wrapped with the `MedianForest` in a single `Pipeline`.

✚ Projects: Common mistake: target-encoding `Agent_name`

`Agent_name` has very high cardinality (hundreds of individual agents), and it was tempting to target-encode it like the other categoricals. In practice, **target-encoding `Agent_name` hurt the score**. The winning configuration keeps it **only as a count feature** — `Agent_name_cnt`, the number of listings that agent posted — computed in `create_shared_stats()`. The count carries the signal (how active an agent is) without letting the model memorise per-agent target means.

🔖 Projects: Tip: cap the target, train on the raw target

A handful of extreme rentals distort the target distribution. The notebook caps the target at **1850 BHD** — rows above the cap are dropped (10448 / 10575 rows kept) — and then trains on the *raw* (uncapped) target so the model still sees the true rental values. Final predictions are clipped to non-negative values before submission.

1.5 Cross-Validation and Submission

The model is validated with **5-fold StratifiedKfold** on log-scaled target bins, reporting the final CV **MAE in BHD**. A fresh pipeline is fitted on the capped data for the test predictions,

and the results are written to `submission_v6.csv`:

📁 Projects: Cross-validation loop on the capped data

```

1 CAP = 1850
2 cap_mask = tr['rent'] <= CAP
3 tr_capped = tr[cap_mask].reset_index(drop=True)
4 y_capped = y[cap_mask]
5
6 strat_labels = pd.qcut(np.log1p(y_capped), 10, labels=False,
7                        duplicates='drop')
8 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=
9                       SEED)
10 CV_preds = np.empty(len(y_capped))
11 for train_idx, val_idx in cv.split(tr_capped, strat_labels):
12     fold_model = build_pipeline()
13     fold_model.fit(tr_capped.iloc[train_idx], y_capped[
14                   train_idx])
15     CV_preds[val_idx] = fold_model.predict(tr_capped.iloc[
16                   val_idx])
17
18 CV_mae = mean_absolute_error(y_capped, CV_preds)
19 print(f"MAE: {CV_mae:.2f} BHD")

```

Experiment summary — what actually worked:

Configuration	Outcome	CV MAE
RandomForest aggregating the <i>mean</i> of trees	baseline aggregation	—
MedianForest (n=800, mf=0.3, msl=1)	median aggregation chosen	80.69 BHD
Manual hierarchical encoding	replaced by TargetEncoder	—
TargetEncoder cv=5	chosen categorical encoding	used
Agent_name target-encoded	hurt the score	worse
Agent_name as count only	final choice	used

With the median-optimised model and the count-only treatment of **Agent_name**, the script reaches a **CV MAE of 80.69 BHD** and writes the final predictions to `submission_v6.csv`, ready for the leaderboard.

► The winning configuration (MedianForest aggregation, TargetEncoder cv=5, groupby aggregation, and Agent_name as count only) achieved a CV MAE of 80.69 BHD, which is the best score for the hard target variable (rent) on the 1850 BHD dataset.